Diseño general del proyecto

Sistema de Gestión Académica - arquitectura de una API CRUD

Objetivo: construir y probar un servicio web que permita crear, consultar, modificar y eliminar registros académicos, almacenados en tablas relacionales.

1. Visual Studio Code

Escribimos código Python y ejecutamos comandos en la terminal.

→

2. FastAPI

Recibe peticiones HTTP y ejecuta las funciones del CRUD.

→

3. SQLite

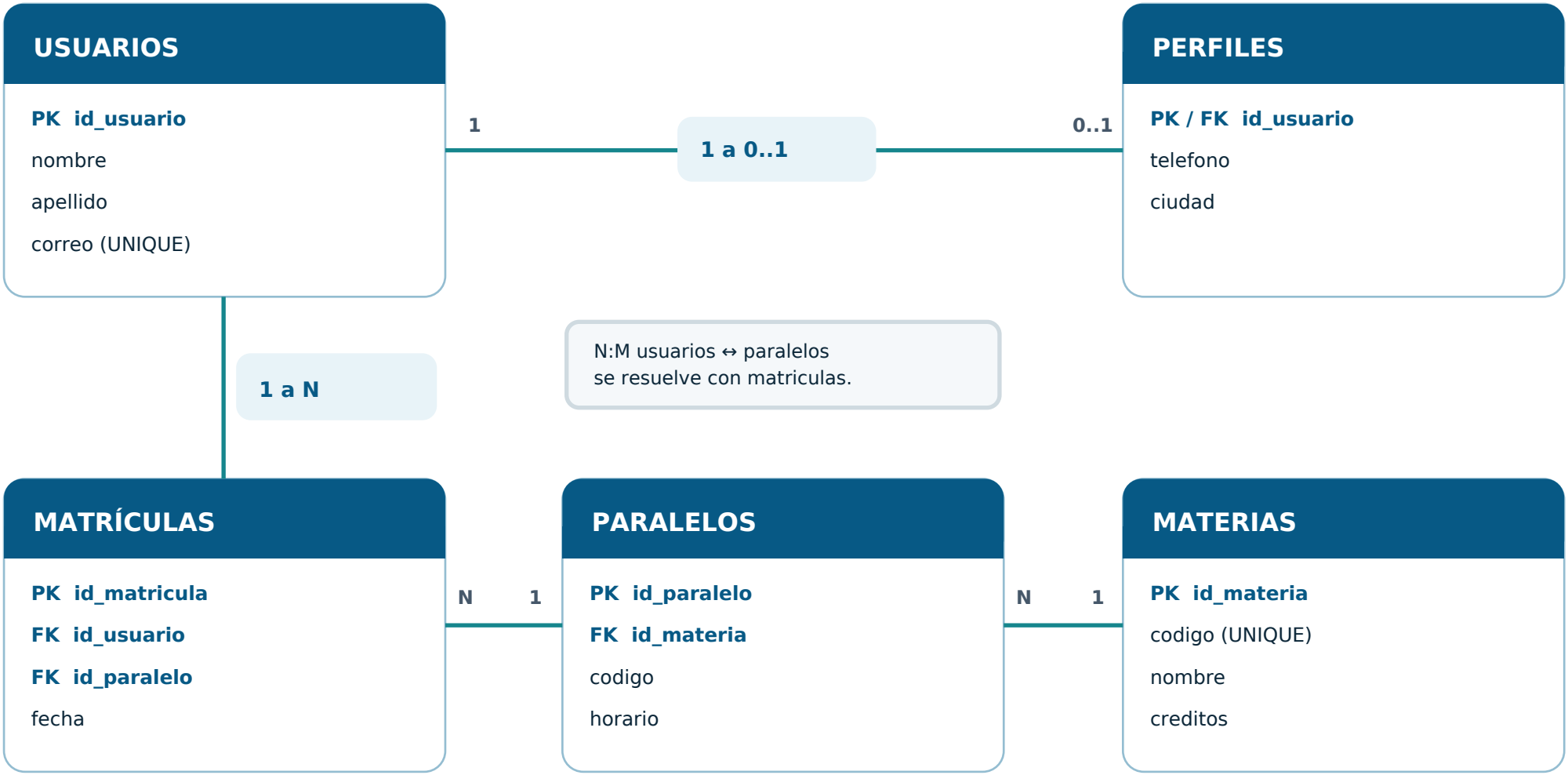
Almacena la información en el archivo universidad.db.

¿Qué se instala y qué NO se necesita?

- Necesario: Python 3.10 o posterior, Visual Studio Code y librerías fastapi/uvicorn.
- SQLite está incluido normalmente con Python a través de sqlite3; no hace falta servidor.
- No es necesario instalar PostgreSQL, MongoDB, Docker, MinIO ni Postman para esta práctica.
- Swagger UI permite ejecutar pruebas desde http://127.0.0.1:8000/docs.

Diagrama entidad-relación

Las claves primarias (PK) identifican registros; las foráneas (FK) establecen relaciones.



Relaciones: usuario–perfil (1:1); materia–paralelos (1:N); usuario–paralelos (N:M, a través de matrículas).

Diseño relacional y normalización

Cada tabla describe una entidad o relación independiente.

Tabla	Clave primaria	Claves foráneas	Descripción
usuarios	id_usuario	—	Datos básicos del estudiante
perfiles	id_usuario	id_usuario → usuarios	Información adicional, 1:1
materias	id_materia	—	Catálogo de materias
paralelos	id_paralelo	id_materia → materias	Grupos que ofrece cada materia
matriculas	id_matricula	id_usuario → usuarios id_paralelo → paralelos	Inscripción en un paralelo

¿Por qué las tablas están normalizadas?

1FN: cada celda contiene un valor, sin listas de materias dentro de una sola columna.

2FN: los datos de cada entidad dependen de su identificador; los perfiles no repiten materias.

3FN: el nombre de la materia se guarda en materias, no se copia a cada paralelo o matrícula. Las tablas se unen mediante sus FK.

Prueba del CRUD y ejecución

Guía breve para demostrar el proyecto en una computadora Windows.

1. Ejecutar desde VS Code

```
py -m venv .venv
```

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

```
.\.venv\Scripts\python.exe -m uvicorn main:app --reload
```

Abre en el navegador: http://127.0.0.1:8000/docs

2. Probar operaciones HTTP

POST	/usuarios	Crear estudiante.
GET	/usuarios	Consultar estudiantes.
PUT	/usuarios/{id_usuario}	Actualizar estudiante.
DELETE	/usuarios/{id_usuario}	Eliminar estudiante.

3. Demostración de relaciones en Swagger

1. Crea un usuario; después crea su perfil usando el mismo id_usuario (1:1).
2. Crea una materia; después un paralelo que incluya su id_materia (1:N).
3. Crea una matrícula con id_usuario e id_paralelo; consulta /reportes/matriculas (N:M).
4. Intenta eliminar una materia con paralelos: la FK impide borrar el registro padre.
5. Para agregar datos ficticios: .\.venv\Scripts\python.exe cargar_ejemplos.py

Nota: esta es una API local para prácticas académicas. El archivo universidad.db se crea automáticamente. Evita registrar datos personales reales sin permiso.